UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias

Introducción a las Ciencias de la Computación

Notas

Profesor:
Salvador López Mendoza
Ayudante:
Kevin Jair Torres Valencia
Ayudante de Laboratorio:
Miriam Torres Bucio

Índice

1. Proceso de programacion **3**
1.1. Planteamiento del problema . . . 3
1.2. Análisis del problema . . . 4
1.3. Diseño de la solución . . . 4
1.3.1. Metodologia de diseno . . . 4
1.4. Codificación . . . 5
1.5. Pruebas y depuración . . . 5
1.6. Documentación . . . 5
1.7. Mantenimiento . . . 6
1.8. Ejemplos . . . 6

2. Creacion y uso de datos primitivos **11**
2.1. Identificadores . . . 11
2.2. Declaracion de datos . . . 11
2.3. Uso de datos (operadores) . . . 12
2.3.1. Operador de asignacion . . . 12
2.3.2. Operadores aritmeticos . . . 12
2.3.3. Operador de asignacion compuesto . . . 13
2.3.4. Operadores de relacion . . . 13
2.3.5. Operadores logicos . . . 13
2.3.6. Operador + para cadenas . . . 13
2.3.7. Precedencia y asociacion de operadores . . . 13
2.4. Conversion de tipo . . . 14
2.5. Ejemplo . . . 14

3. Creacion y uso de objetos **15**
3.1. Creacion de objetos . . . 15
3.2. Uso de objetos . . . 15
3.3. Eliminacion de objetos . . . 15
3.4. Objetos para lectura y escritura . . . 15
3.5. Ejemplos de trabajo con objetos . . . 16

1. Proceso de programacion

El proceso de programación consiste en un conjunto de actividades necesarias para desarrollar programas que resuelvan correctamente un problema específico. Estas actividades incluyen: definición del problema, diseño de la solución, implementación, depuración, pruebas, documentación y mantenimiento.

El proceso no es lineal sino **iterativo e incremental**. Durante el desarrollo es común regresar a etapas anteriores para mejorar o corregir la solución.

Programar implica más que escribir código. Antes de implementar una solución es necesario comprender el problema y diseñar una estrategia para resolverlo.

Las principales etapas del proceso de programación son:

- **Planteamiento del problema**.
- **Análisis del problema**.
- **Diseño de la solución**.
- **Codificación**.
- **Pruebas y depuración**.
- **Documentación**.
- **Mantenimiento**.

La documentación se realiza durante todo el proceso para registrar cómo y por qué se construyó la solución.

1.1. Planteamiento del problema

En esta etapa se determina con precisión **qué debe hacer el programa**. Una definición incorrecta o incompleta del problema conduce frecuentemente a soluciones inadecuadas.

Para definir el problema se deben identificar:

- El objetivo del programa.
- Los requerimientos del usuario.
- Las características que debe cumplir la solución.

Los requerimientos iniciales suelen ser ambiguos o incompletos, por lo que el programador debe analizarlos y refinarlos hasta eliminar ambigüedades.

1.2. Análisis del problema

Consiste en descomponer el problema en partes más pequeñas y manejables. Se analizan las relaciones entre los datos de entrada, los procesos necesarios y los resultados que se deben obtener.

En esta etapa **no se programa**, únicamente se razona la solución.

1.3. Diseño de la solución

El diseño consiste en definir una estrategia para resolver el problema mediante un programa.

Un buen diseño permite:

- organizar la solución del problema,
- identificar los componentes principales,
- establecer relaciones entre ellos,
- facilitar modificaciones futuras.

Existen diferentes formas de diseñar una solución.

1.3.1. Metodologia de diseno

Una forma tradicional de diseño es describir paso a paso el procedimiento para resolver el problema.

Este procedimiento se conoce como **algoritmo**.

Un algoritmo es una descripción ordenada de los pasos necesarios para resolver un problema, con instrucciones claras y sin ambigüedades, que produce siempre el mismo resultado si se ejecuta bajo las mismas condiciones.

Los algoritmos pueden expresarse mediante:

- pseudocódigo,
- diagramas,
- o directamente mediante un programa.

En la programación orientada a objetos el diseño también implica identificar **objetos**, sus responsabilidades y la forma en que interactúan mediante mensajes.

1.4. Codificación

Consiste en traducir el diseño a un lenguaje de programación, por ejemplo Java. En esta etapa se escribe el código fuente del programa siguiendo el algoritmo o modelo diseñado previamente.

Se debe cuidar el uso correcto de:

- La sintaxis del lenguaje
- Los tipos de datos
- Las estructuras de control

Posteriormente el programa se **compila** y se **ejecuta** para verificar su funcionamiento.

1.5. Pruebas y depuración

La depuración es el proceso de localizar y corregir errores en un programa. Los errores pueden ser de distintos tipos:

- errores de sintaxis,
- errores de ejecución,
- errores lógicos.

Durante la depuración se revisa el código y se realizan pruebas para asegurar que el programa funcione correctamente.

1.6. Documentación

La documentación consiste en registrar información sobre el programa, incluyendo:

- descripción del problema,
- diseño de la solución,
- funcionamiento del programa,
- instrucciones para su uso.

Una buena documentación facilita el mantenimiento y la comprensión del sistema por parte de otros programadores.

1.7. Mantenimiento

El mantenimiento consiste en realizar modificaciones al programa después de su desarrollo inicial. Estas modificaciones pueden realizarse para:

- agregar nuevas funcionalidades,
- mejorar el desempeño,
- corregir errores detectados posteriormente.

El mantenimiento es una etapa importante porque la mayoría de los programas evolucionan con el tiempo.

1.8. Ejemplos

Ejemplo 1: Cálculo del promedio de tres calificaciones

1. Planteamiento del problema

Desarrollar un programa que solicite tres calificaciones de un alumno y calcule su promedio. El programa debe indicar si el alumno está aprobado (promedio mayor o igual a 6) o reprobado.

2. Análisis del problema

Entradas:

- Calificación 1
- Calificación 2
- Calificación 3

Proceso:

- Sumar las tres calificaciones
- Dividir entre 3
- Evaluar si el promedio es mayor o igual a 6

Salida:

- Promedio
- Mensaje de aprobado o reprobado

3. Diseño de la solución (Algoritmo en pseudocódigo)

```
Inicio
  Leer cal1, cal2, cal3
  promedio = (cal1 + cal2 + cal3) / 3
  Mostrar promedio
  Si promedio >= 6 entonces
      Mostrar "Aprobado"
  Sino
      Mostrar "Reprobado"
Fin
```

4. Codificación en Java

```java
import java.util.Scanner;

public class PromedioAlumno {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double cal1, cal2, cal3, promedio;

        System.out.print("Ingrese la primera calificación: ");
        cal1 = sc.nextDouble();

        System.out.print("Ingrese la segunda calificación: ");
        cal2 = sc.nextDouble();

        System.out.print("Ingrese la tercera calificación: ");
        cal3 = sc.nextDouble();

        promedio = (cal1 + cal2 + cal3) / 3;

        System.out.println("Promedio: " + promedio);

        if (promedio >= 6) {
            System.out.println("Aprobado");
        } else {
            System.out.println("Reprobado");
        }

        sc.close();
    }
}
```

5. Pruebas

Entrada: 8, 7, 9
Salida esperada: Promedio = 8.0 → Aprobado
Entrada: 5, 6, 4
Salida esperada: Promedio = 5.0 → Reprobado

6. Documentación

El programa calcula el promedio de tres calificaciones y determina la situación académica del alumno.

7. Mantenimiento

Podría modificarse para permitir cualquier número de calificaciones usando ciclos.

Ejemplo 2: Determinar si un número es par o impar

1. Planteamiento del problema

Desarrollar un programa que determine si un número entero ingresado por el usuario es par o impar.

2. Análisis del problema

Entrada:

- Número entero

Proceso:

- Obtener el residuo de dividir el número entre 2
- Si el residuo es 0, el número es par
- En otro caso, es impar

Salida:

- Mensaje indicando si es par o impar

3. Diseño de la solución (Algoritmo)

```
Inicio
  Leer numero
  Si numero mod 2 = 0 entonces
      Mostrar "Es par"
  Sino
      Mostrar "Es impar"
Fin
```

4. Codificación en Java

```
import java.util.Scanner;

public class NumeroParImpar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int numero;

        System.out.print("Ingrese un número entero: ");
        numero = sc.nextInt();

        if (numero % 2 == 0) {
            System.out.println("El número es par");
        } else {
            System.out.println("El número es impar");
        }

        sc.close();
    }
}
```

5. Pruebas

Entrada: 10 → Salida: Es par
Entrada: 7 → Salida: Es impar

6. Documentación

El programa determina la paridad de un número utilizando el operador módulo.

7. Mantenimiento

Podría ampliarse para validar que el usuario no ingrese caracteres no numéricos.

Ejemplo 3: Conversión de grados Celsius a Fahrenheit

1. Planteamiento del problema

Desarrollar un programa que convierta una temperatura dada en grados Celsius a grados Fahrenheit.

2. Análisis del problema

Entrada:

- Temperatura en grados Celsius

Proceso: La fórmula de conversión es:

$$F = \frac{9}{5}C + 32$$

Salida:

- Temperatura en grados Fahrenheit

3. Diseño de la solución (Algoritmo)

```
Inicio
  Leer C
  F = (9/5) * C + 32
  Mostrar F
Fin
```

4. Codificación en Java

```java
import java.util.Scanner;

public class ConversionTemperatura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double celsius, fahrenheit;

        System.out.print("Ingrese la temperatura en Celsius: ");
        celsius = sc.nextDouble();

        fahrenheit = (9.0 / 5.0) * celsius + 32;

        System.out.println("Temperatura en Fahrenheit: " + fahrenheit);

        sc.close();
    }
}
```

5. Pruebas

Entrada: 0
Salida esperada: 32

6. Documentación

El programa realiza una conversión directa usando una fórmula matemática sin estructuras de control.

7. Mantenimiento

Podría ampliarse para convertir también de Fahrenheit a Celsius.

2. Creacion y uso de datos primitivos

Los **datos primitivos** son tipos de datos predefinidos por el lenguaje Java que permiten representar valores simples como números o valores lógicos.

Estos tipos se utilizan para almacenar información básica y realizar operaciones mediante expresiones.

2.1. Identificadores

Un identificador es el nombre que se asigna a variables, métodos o clases. Para definir identificadores se deben seguir ciertas reglas:

- Deben comenzar con una letra, guion bajo.
- No pueden comenzar con un número.
- No pueden coincidir con palabras reservadas del lenguaje.

Los identificadores deben elegirse de forma que describan claramente su propósito. Existe un conjunto de palabras que no pueden ser usadas como identificadores porque tiene un significado especial para java, cada una de éstas se denomina palabra reservada.

2.2. Declaracion de datos

La declaración de una variable consiste en especificar el tipo de dato que almacenará y el identificador que la representará.
La forma general es:

```
tipo identificador;
```

También es posible inicializar la variable en el momento de declararla:

```
tipo identificador = valor;
```

Los tipos de datos definidos en java, y denominados primitivos, son:

- Tipo numérico (enteros o reales).
- Tipo carácter.
- Tipo Booleano.

Tipo	**Descripción**
`byte`	Entero de 8 bits
`short`	Entero de 16 bits
`int`	Entero de 32 bits
`long`	Entero de 64 bits
`float`	Real en 32 bits con 7 dígitos en la parte decimal
`double`	Real en 64 bits con 15 dígitos en la parte decimal
`char`	Carácter en 16-bits (Unicode)
`boolean`	Booleano

2.3. Uso de datos (operadores)

Los datos primitivos se utilizan principalmente en expresiones que emplean operadores.

2.3.1. Operador de asignacion

El operador de asignación (`=`) se utiliza para almacenar un valor en una variable.

```
x = 5;
```

Esto indica que el valor 5 se asigna a la variable `x`.

2.3.2. Operadores aritmeticos

Los operadores aritméticos permiten realizar operaciones matemáticas.

Operador	**Descripción**
`+`	Suma
`-`	Resta
`*`	Multiplicación
`/`	División
`%`	Residuo de la división

2.3.3. Operador de asignacion compuesto

Los operadores de asignación compuesta combinan una operación aritmética con una asignación.
Ejemplos:

```
x += 5;
x -= 3;
x *= 2;
x /= 4;
```

2.3.4. Operadores de relacion

Los operadores de relación permiten comparar valores y producen un resultado booleano.

- `==`: Asignacion.
- `!=`: Negacion.
- `<`: Menor.
- `>`: Mayor.
- `<=`: Menor igual que.
- `>=`: Mayor igual que.

2.3.5. Operadores logicos

Los operadores lógicos se utilizan para combinar expresiones booleanas.

- AND lógico: `&&`
- OR lógico: `||`
- NOT lógico: `!`

2.3.6. Operador + para cadenas

El operador `+` también puede utilizarse para concatenar cadenas de texto.

```
String mensaje = "Hola " + nombre;
```

2.3.7. Precedencia y asociacion de operadores

Cuando una expresión contiene varios operadores, Java evalúa la expresión siguiendo reglas de **precedencia** y **asociación**.
Estas reglas determinan el orden en que se ejecutan las operaciones.

2.4. Conversion de tipo

La conversión de tipo permite transformar un valor de un tipo de dato a otro.
Puede ser:

- **implícita**: realizada automáticamente por el compilador.
- **explícita**: indicada por el programador mediante casting.

Ejemplo de conversión implícita:

```
int x = 42;
double numeroDecimal = x;
```

Ejemplo de conversión explícita:

```
int x = (int) 3.5;
```

2.5. Ejemplo

```
public class DatosPrimitivos {

    public static void main(String[] args) {

        int edad = 21;
        double estatura = 1.75;
        char grupo = 'A';
        boolean inscrito = true;

        System.out.println("Edad: " + edad);
        System.out.println("Estatura: " + estatura);
        System.out.println("Grupo: " + grupo);
        System.out.println("Inscrito: " + inscrito);

        double edadFutura = edad + 5;

        System.out.println("Edad en 5 años: " + edadFutura);
    }
}
```

3. Creacion y uso de objetos

En la programación orientada a objetos, los programas se construyen mediante la interacción de objetos.

Un **objeto** representa una entidad del problema que se desea modelar y posee:

- **estructura**: atributos o datos que describen al objeto,
- **comportamiento**: operaciones que el objeto puede realizar,
- **identidad**: característica que distingue a un objeto de otro.

3.1. Creacion de objetos

Los objetos se crean a partir de una clase utilizando el operador `new`.
Ejemplo:

```
Scanner entrada = new Scanner(System.in);
```

La variable almacena una **referencia** al objeto creado.

3.2. Uso de objetos

Los objetos interactúan entre sí mediante el envío de **mensajes**. En Java, un mensaje corresponde a la invocación de un método.
Ejemplo:

```
entrada.nextInt();
```

El método se ejecuta sobre el objeto indicado por la referencia.

3.3. Eliminacion de objetos

Cuando un objeto deja de ser utilizado y no existen referencias hacia él, el sistema puede liberar la memoria que ocupa.

Este proceso es gestionado automáticamente por el **recolector de basura** (garbage collector).

3.4. Objetos para lectura y escritura

Java proporciona clases para facilitar la interacción con el usuario. Una de las más utilizadas es la clase `Scanner`, que permite leer datos desde la entrada estándar.
Ejemplo:

```
Scanner sc = new Scanner(System.in);
int numero = sc.nextInt();
```

3.5. Ejemplos de trabajo con objetos

El trabajo con objetos incluye operaciones como:

- creación de objetos,
- envío de mensajes (llamada a métodos),
- recepción de valores devueltos por métodos,
- interacción entre varios objetos.

La colaboración entre objetos permite dividir un problema complejo en componentes más simples.